

Exercise 7:

Recursion is a technique where a function calls itself. It simplifies problems by dividing problems into smaller problems. It's stopped by base case. Until it reaches the base case, it is recursively called on smaller problems.

Code:

```
1  import java.util.*;
2
3  public class FinancialForecast {
4      static double calcFV(double PV, double g, int n){
5          if (n==0)
6              return PV;
7          else
8              return calcFV(PV, g, n-1) * (1+g);
9      }
10
11     Run | Debug
12     public static void main(String[] args){
13         Scanner sc = new Scanner(System.in);
14
15         double presentValue = 1000.0;
16         double growthRate = 0.02;
17         int years = 5;
18
19         System.out.println("Present Value: Rs." + presentValue);
20         System.out.println("Growth Rate: " + growthRate*100 + "%");
21         System.out.println("Years: " + years);
22
23         System.out.printf("Future Value: %.2f\n", calcFV(presentValue, growthRate, years));
24     }
25 }
```

Output:

```
Present Value: Rs.1000.0
Growth Rate: 2.0%
Years: 5
Future Value: 1104.08
```

For each year, a recursive call is made so time complexity is $O(n)$.

If too many recursive calls are made, it can be a problem. Thus, if iterative approach is similar in time complexity (or better), it is best to use that to avoid too many calls.